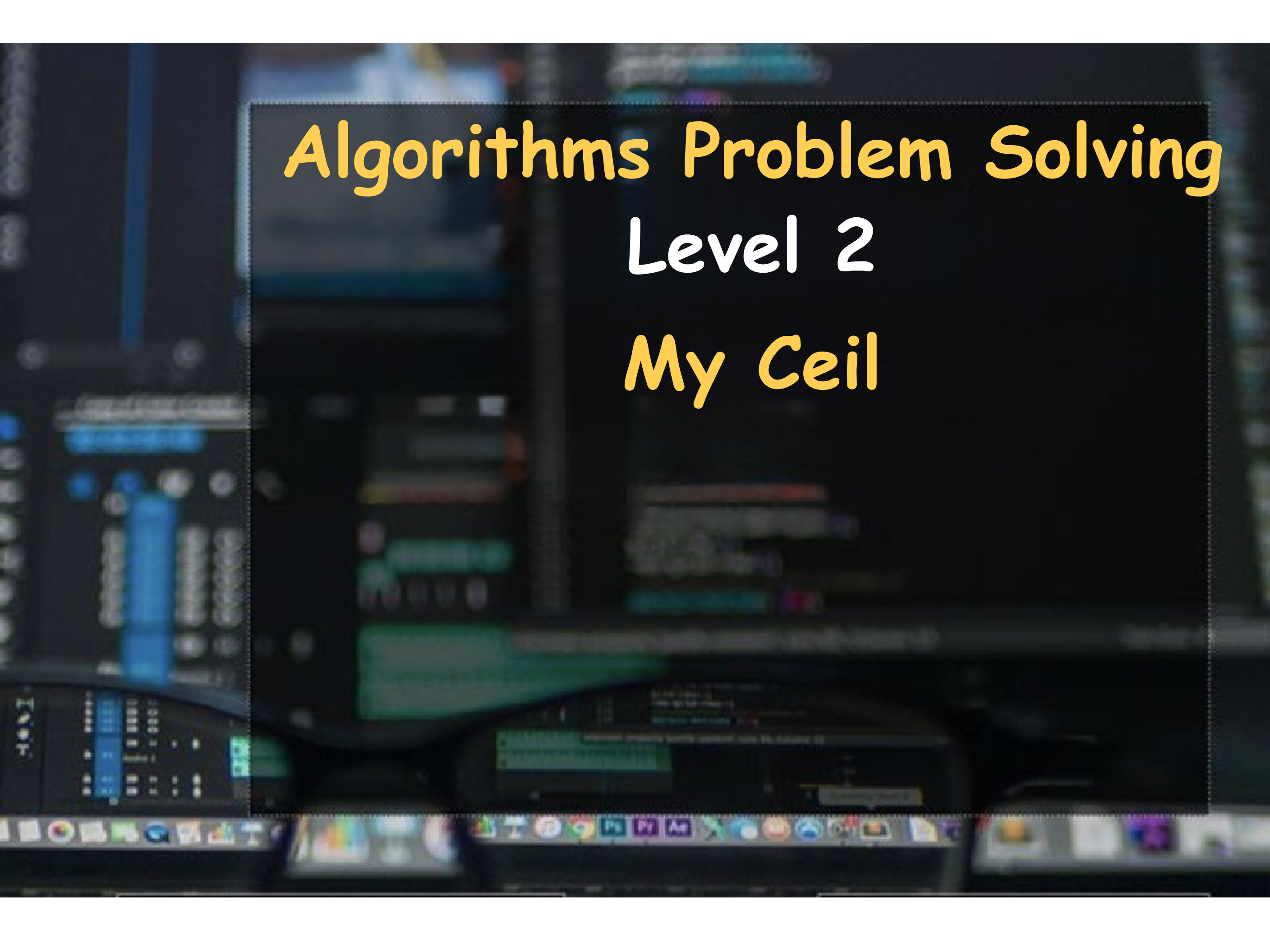
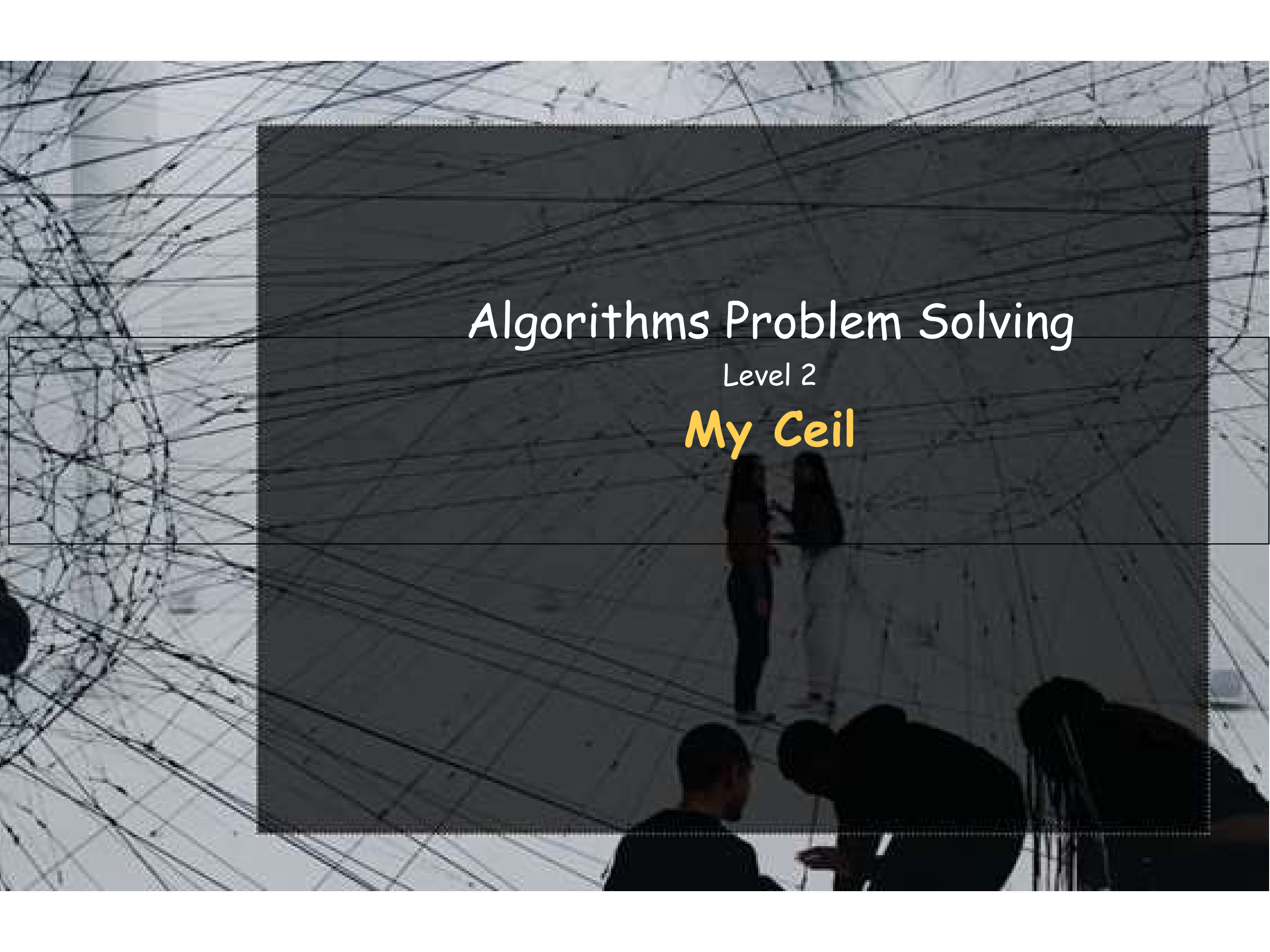




Algorithms Problem Solving

Level 2

My Ceil

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the foreground, the silhouettes of several people are visible, some standing and some sitting, looking up at the ceiling. The overall atmosphere is one of architectural elegance and openness.

Algorithms Problem Solving

Level 2

My Ceil

Definition

- ◆ This program calculates the ceiling value of a number without using the built-in ``ceil()`` function.
- ◆ The ceiling of a number is the smallest integer that is greater than or equal to the number.
- ◆ If the number already has no fractional part, it remains unchanged.
- ◆ If there is a fractional part, the number moves to the next integer.

Explanation

- ◆ ``ReadNumber()`` reads a floating-point number from the user.
- ◆ ``GetFractionPart()`` extracts the fractional part:
`return number - int(number);`
- ◆ For example:
 $33.54323 - 33 = 0.54323$
- ◆ ``MyCeil()`` first stores the integer part:
`int integerPart = number;`
- ◆ It then gets the fractional part.
- ◆ The condition:
`if (abs(FractionPart) > 0)`
checks whether the number has a fractional part.
- ◆ If the number is positive and has a fractional part, the integer part is increased by ``1``.
- ◆ If the number is negative and has a fractional part, the integer part is decreased by ``1``.
- ◆ This handles the difference between truncation toward zero and the mathematical ceiling operation.

☐ Steps in main()

- ☐ Ask the user to enter a floating-point number.
- ☐ Store the integer part of the number.
- ☐ Calculate the fractional part.
- ☐ Check whether the fractional part is different from `0`.
- ☐ If positive, increase the integer part by `1`.
- ☐ If negative, decrease the integer part by `1`.
- ☐ If there is no fractional part, return the integer part unchanged.
- ☐ Print the result from `MyCeil()`.
- ☐ Compare it with C++'s built-in `ceil()` function.

💡 Example

◆ Input:

33.54323

◆ Integer part:

33

◆ Fractional part:

$33.54323 - 33 = 0.54323$

◆ Since the fractional part is greater than `0`, the number is positive, so:

$33 + 1 = 34$

◆ Therefore:

$\text{MyCeil}(33.54323) = 34$

◆ For a negative number:

-33.54323

◆ Integer part:

-33

◆ Fractional part:

$-33.54323 - (-33) = -0.54323$

◆ Since the number is negative and has a fractional part:

$-33 - 1 = -34$

◆ This is important: `-34` is the ceiling of `-33.54323`? Actually, no. The mathematical ceiling is `-33`.

Note

- ◆ There is an important bug in the provided `MyCeil()` function for negative decimal numbers.
- ◆ For example:
-33.54323
- ◆ The correct ceiling is:
-33
- ◆ But the current code does:
`return --integerPart;`
- ◆ This returns `-34`, which is actually the floor.
- ◆ For a correct `MyCeil()`, negative numbers with a fractional part should return the integer part unchanged

⊘ Warning

- ◆ The current code works for positive decimal numbers such as `33.54323`.
- ◆ It is incorrect for negative decimal numbers.
- ◆ `MyCeil(-33.54323)` currently returns `-34`, while C++ `ceil(-33.54323)` returns `-33`.
- ◆ This happens because `int(number)` truncates toward zero.
- ◆ `cmath` is used for `abs()` and the built-in `ceil()` comparison.
- ◆ `corecrt\_math\_defines.h` is not needed.

Comparison

`MyCeil()`

- ✓ User-defined function.
- ✓ Does not use the built-in `ceil()` function.
- ✓ For positive decimal numbers, increases the integer part by `1`.
- ✓ The provided version needs correction for negative decimal numbers.

`ceil()`

- ✓ Built-in C++ function.
- ✓ Directly calculates the ceiling value.
- ✓ Correctly handles positive and negative numbers.
- ✓ Used in this program to compare the result with `MyCeil()`.

✦ Summary

- ✓ Read a floating-point number.
- ✓ Extract the integer and fractional parts.
- ✓ If there is no fractional part, return the integer unchanged.
- ✓ For positive decimal numbers, increase the integer part by `1`.
- ✓ For negative decimal numbers, keep the integer part unchanged.
- ✓ The provided code has a bug for negative decimal numbers.
- ✓ Compare the custom result with C++'s `ceil()` function.

Thank you 😊